LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 1

Struct dan Pointer

Oleh:

Muhammad Kurniawan Pasya NIM. 2510817210026

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL
2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 1

Laporan Praktikum Algoritma & Struktur Data Modul 1: Struct dan Pointer ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Muhammad Kurniawan Pasya
NIM : 2510817210026

Menyetujui,	Mengetahui,
Asisten Praktikum	Dosen Penanggung Jawab Praktikum
Muhammad Fauzan Ahsani	Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIM. 2310817310009	NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN 2

DAFTAR ISI 3

DAFTAR GAMBAR 4

DAFTAR TABEL 5

SOAL 1 6

A. Source Code 6

B. Output Program 8

C. Pembahasan 8

SOAL 2 14

A. Source Code 14

TAUTAN GIT 23

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1 8

DAFTAR TABEL

Tabel 1 Source Code prob1.cpp 6
Tabel 2 Source Code line.cpp 6
Tabel 3 Source Code line.h 7
Tabel 4 Source Code point.cpp 7
Tabel 5 Source Code point.h 7

SOAL 1

Diberikan sebuah garis l yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik $P = (x_P, y_P)$. Tentukan posisi titik P terhadap garis l dengan cara mengimplementasikan fungsi gradient dan CheckPointPosition pada file header yang telah diberikan.

Input	**Output**
`1 0 -3 5 0`	`Left`
`1 0 -3 1 7`	`Right`
`2 -1 -1 1 1`	`On Line`

A. Source Code

Tabel 1 Source Code prob1.cpp

```
#include <iostream>
#include "line.h"
#include "point.h"

int main() {
    Line l;
    Point p;

    while (std::cin >> l.a >> l.b >> l.c >> p.x >> p.y) {
        double val = gradient(&l, &p);
        std::cout << CheckPointPosition(val) << std::endl;
    }

    return 0;
}
```

Tabel 2 Source Code line.cpp

```
#include <iostream>
#include "line.h"
```

```
#include "point.h"

int main() {
    Line l;
    Point p;

    while (std::cin >> l.a >> l.b >> l.c >> p.x >> p.y) {
        double val = gradient(&l, &p);
        std::cout    <<    CheckPointPosition(val)    <<
std::endl;
    }

    return 0;
}
```

Tabel 3 Source Code line.h

```
#ifndef LINE_H
#define LINE_H

#include "point.h"
#include <string>

struct Line {
    double a;
    double b;
    double c;
};

inline constexpr double EPSILON = 1e-9;

double gradient(const Line * l,const Point * p);

std::string CheckPointPosition(double gradient);

#endif
```

Tabel 4 Source Code point.cpp

```
#include "point.h"
```

Tabel 5 Source Code point.h

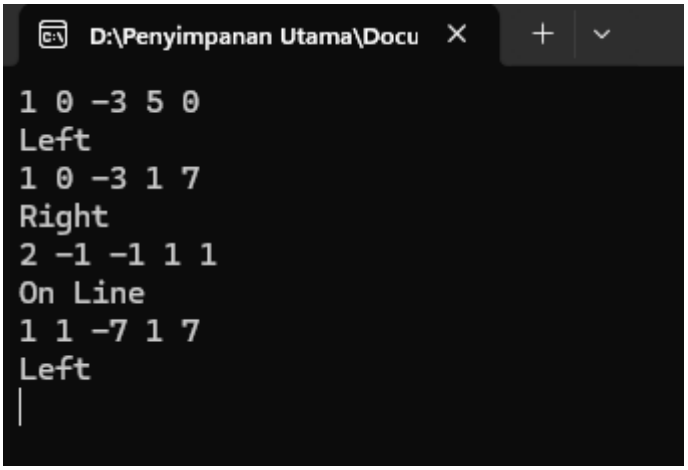
```
#ifndef POINT_H
```

```
#define POINT_H

struct Point {
    double x;
    double y;
};


#endif
```

B. Output Program



Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Pertama-tama, program ini bertujuan untuk menentukan posisi suatu titik terhadap sebuah garis dalam bidang dua dimensi.

Di soal dikasih garis

$$ax + by + c = 0$$

Titik

$$xp, yp$$

Terus nentuin posisi titik terhadap garis

Left, Right, On Line

Berarti masukan titik ke persamaan garis

$$Value = a.xp + b.yp + c$$

Lalu jika

Value = 0 , artinya On Line

Value > 0 , satu sisi

Value < 0 , sisi lain

Di line.h

```
double gradient(const Line * l,const Point * p);
```

```
std::string CheckPointPosition(double gradient);
```

`gradient()` = hitung nilai tadi

`CheckPointPosition()` = ubah jadi "Left / Right / On Line"

Contoh input kita memakai yg On Line tadi, yaitu

```
2 -1 -1 1 1
```

Kalo memakai persamaan garis tadi, maka jadi:

2*1 + (-1)*1 + (-1)

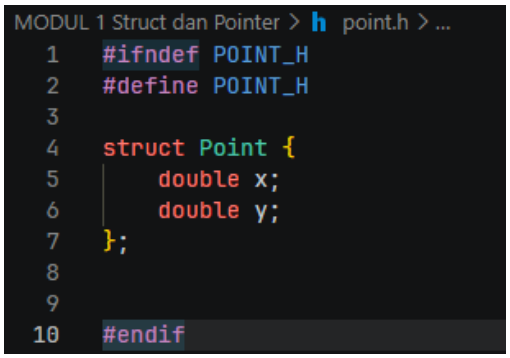
2 + (-1) + (-1) = 1 + (-1)

= 0

Yang artinya ini 0, di mana Value = 0 tadi berada dalam garis, On Line.

Program ini menggunakan konsep modularisasi dengan memanfaatkan beberapa file, yaitu file header (.h) dan file implementasi (.cpp). Header file digunakan untuk mendeklarasikan struktur dan fungsi agar dapat digunakan oleh file lain, sedangkan file .cpp berisi implementasi dari fungsi tersebut. Pendekatan ini membuat kode lebih terstruktur, mudah dipahami, dan dapat digunakan kembali.

File point.h digunakan untuk mendefinisikan struktur Point yang merepresentasikan koordinat titik dalam bidang dua dimensi. Struktur ini memiliki dua atribut, yaitu x dan y, yang menyimpan posisi titik. Struktur ini menjadi komponen penting karena digunakan sebagai input dalam proses perhitungan posisi titik terhadap garis.



Gambar 2. point.h

Selanjutnya, file line.h mendefinisikan struktur Line yang berisi parameter a, b, dan c dari persamaan garis. Selain itu, pada file ini juga dideklarasikan fungsi gradient dan `CheckPointPosition`. Fungsi `gradient` digunakan untuk menghitung nilai substitusi titik ke dalam persamaan garis, sedangkan fungsi `CheckPointPosition` digunakan untuk menentukan posisi titik berdasarkan hasil perhitungan tersebut.

```cpp
#ifndef LINE_H
#define LINE_H

#include "point.h"
#include <string>

struct Line {
    double a;
    double b;
    double c;
};

inline constexpr double EPSILON = 1e-9;

double gradient(const Line * l,const Point * p);

std::string CheckPointPosition(double gradient);

#endif
```

Gambar 3. line.h

Fungsi `gradient` yang ada di line.cpp bertugas menghitung nilai dari persamaan garis dengan memasukkan koordinat titik. Hasil dari fungsi ini berupa angka yang kemudian akan diproses lebih lanjut. Penggunaan pointer pada parameter bertujuan agar data bisa dikirim dengan lebih efisien tanpa perlu disalin.

```cpp
#include "line.h"
#include <cmath>

double gradient(const Line * l,const Point * p) {
    return l->a * p->x + l->b * p->y + l->c;
}

std::string CheckPointPosition(double value) {
    if (std::abs(value) < EPSILON) {
        return "On Line";
    } else if (value > 0) {
        return "Left";
    } else {
        return "Right";
    }
}
```

Gambar 4. line.cpp

Fungsi `CheckPointPosition` digunakan untuk menentukan posisi titik. Fungsi ini akan mengecek apakah nilai hasil perhitungan mendekati nol, lebih besar dari nol, atau lebih kecil dari nol. Untuk menghindari kesalahan karena bilangan desimal, digunakan nilai toleransi kecil yang disebut EPSILON.

File prob1.cpp merupakan program utama yang menjalankan semuanya. Program ini membaca input dari pengguna, memanggil fungsi untuk menghitung nilai, lalu menentukan posisi titik, dan akhirnya menampilkan hasilnya.

Secara keseluruhan, program ini menunjukkan cara sederhana untuk menentukan posisi titik terhadap garis menggunakan perhitungan matematika dasar. Selain itu, pembagian program ke dalam beberapa file juga membantu membuat kode lebih rapi, mudah dipahami, dan lebih mudah dikembangkan.

Terakhir ada prob1.cpp, bisa dilihat kodenya di source code atas.

File `prob1.cpp` merupakan program utama yang digunakan untuk menjalankan seluruh proses pada soal pertama. Di dalam file ini terdapat fungsi `main()` yang menjadi titik awal eksekusi program.

Pada awal program, dideklarasikan dua variabel, yaitu `Line l` untuk menyimpan data garis (nilai a, b, dan c), serta `Point p` untuk menyimpan koordinat titik yang akan diuji.

Program kemudian menggunakan perulangan `while` untuk membaca input dari pengguna. Data yang dimasukkan terdiri dari nilai a, b, dan c dari persamaan garis, serta koordinat titik (x, y). Dengan menggunakan perulangan ini, program dapat memproses beberapa data sekaligus selama masih ada input.

Setelah data dibaca, program memanggil fungsi `gradient` dengan mengirimkan alamat variabel garis dan titik. Fungsi ini akan menghitung nilai dari persamaan garis dengan memasukkan koordinat titik ke dalam rumus:

$$ax + by + c$$

Hasil dari perhitungan tersebut kemudian disimpan dalam sebuah variabel. Nilai ini menunjukkan posisi relatif titik terhadap garis.

Selanjutnya, nilai tersebut dikirim ke fungsi `CheckPointPosition` untuk menentukan posisi titik. Fungsi ini akan mengembalikan hasil berupa teks, yaitu “Left”, “Right”, atau “On Line”, tergantung dari nilai yang dihasilkan.

Hasil akhir kemudian ditampilkan ke layar menggunakan `std::cout`. Setiap hasil ditampilkan pada baris baru agar mudah dibaca.

Program akan terus berjalan selama masih ada input yang diberikan. Jika input berhenti, maka perulangan akan selesai dan program akan berakhir.

Lalu selanjutnya, semua kode tadi dijalankan perintah:

```
g++ -std=c++23 -Wall -Wextra -o prob1.exe prob1.cpp line.cpp point.cpp
```

g++ : Ini adalah compiler C++ buat ngubah kode .cpp jadi program yang bisa dijalankan.

-std=c++23 : pakai standar C++ versi terbaru (C++23)

-Wall : Aktifin semua warning dasar

Contoh: variabel gak kepakai, typo kecil, potensi bug

-Wextra : Tambahan warning lebih detail

-o prob1.exe : hasil compile bakal jadi file bernama prob1.exe dan di sini lah tadi programnya dijalankan

prob1.cpp line.cpp point.cpp : Ini semua file tadi yang mau dicompile

SOAL 2

Diberikan sebuah lingkaran C dengan pusat (c_x, c_y) dan jari-jari r > 0, serta sebuah titik $P = (p_x, p_y)$. Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi distance dan CheckPointInCircle pada file header yang diberikan.

Input	**Output**
`0 0 5 3 4`	`On Circle`
`0 0 5 1 1`	`Inside`
`0 0 5 4 4`	`Outside`
`-2 -2 4 2 -2`	`On Circle`

A. Source Code

Tabel 6 Source Code prob2.cpp

```
#include <iostream>
#include "line.h"
#include "point.h"

int main() {
    Line l;
    Point p;

    while (std::cin >> l.a >> l.b >> l.c >> p.x >> p.y) {
        double val = gradient(&l, &p);
        std::cout    <<    CheckPointPosition(val)    <<
std::endl;
    }

    return 0;
}
```

Tabel 7 Source Code circle.cpp

```
#include "circle.h"
```

```
 2 #include <cmath>
 3
 4 static double savedRadius;
 5
 6 double distance(const Circle * c,const Point * p) {
 7     savedRadius = c->radius;
 8
 9     return std::sqrt(
10         (p->x - c->centre.x) * (p->x - c->centre.x) +
11         (p->y - c->centre.y) * (p->y - c->centre.y)
    );
12 }
13
14 std::string CheckPointInCircle(double d) {
15     if (std::abs(d - savedRadius) < EPSILON)
16         return "On Circle";
17     else if (d < savedRadius)
18         return "Inside";
19     else
20         return "Outside";
21 }
22
```

Tabel 8 Source Code circle.h

```
 1 #ifndef CIRCLE_H
 2 #define CIRCLE_H
 3
 4 #include "point.h"
 5 #include <string>
 6
 7 struct Circle {
 8     Point centre;
 9     double radius;
10 };
11
12 inline constexpr double EPSILON = 1e-9;
13
14 double distance(const Circle * c,const Point * p);
15
16
17 std::string CheckPointInCircle(double distance);
18
19 #endif
```

Tabel 9 Source Code point.cpp

```
#include "point.h"
```

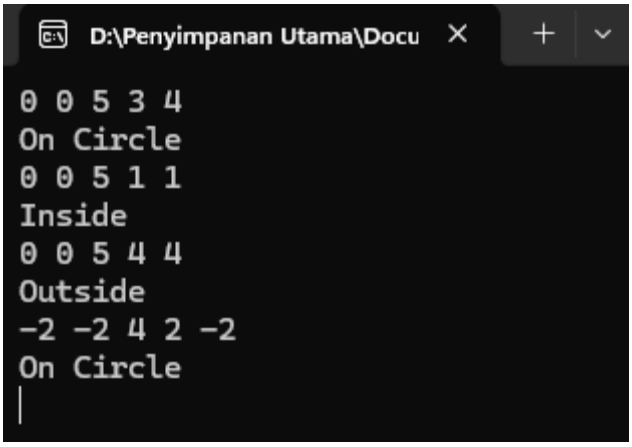
Tabel 10 Source Code point.h

```
#ifndef POINT_H
#define POINT_H

struct Point {
    double x;
    double y;
};


#endif
```

B. Output Program



Gambar 5. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Program ini bertujuan untuk menentukan posisi suatu titik terhadap sebuah lingkaran pada bidang dua dimensi. Lingkaran direpresentasikan dengan pusat (x_c, y_c)dan jari-jari r, sedangkan titik dinyatakan sebagai (x_p, y_p). Hasil yang ingin diperoleh adalah apakah titik berada di dalam lingkaran, tepat pada lingkaran, atau di luar lingkaran.

Untuk menentukan posisi titik, digunakan rumus jarak antara dua titik dalam bidang kartesius, yaitu:

$$d = \sqrt{(x_p - x_c)^2 + (y_p - y_c)^2}$$

Nilai d merupakan jarak antara titik dan pusat lingkaran. Nilai ini kemudian dibandingkan dengan jari-jari lingkaran.

Berdasarkan perbandingan tersebut, posisi titik dapat ditentukan sebagai berikut:

- Jika $d = r$, maka titik berada tepat pada lingkaran (**On Circle**)
- Jika $d < r$, maka titik berada di dalam lingkaran (**Inside**)
- Jika $d > r$, maka titik berada di luar lingkaran (**Outside**)

Karena perhitungan menggunakan bilangan desimal, digunakan nilai toleransi kecil (EPSILON) untuk mengecek apakah dua nilai dianggap sama. Hal ini penting untuk menghindari kesalahan akibat pembulatan angka.

Program ini disusun menggunakan beberapa file agar lebih terstruktur. File header (.h) digunakan untuk mendeklarasikan struktur dan fungsi, sedangkan file .cpp digunakan untuk implementasi. Dengan cara ini, program menjadi lebih rapi dan mudah dipahami.

File `point.h` digunakan untuk mendefinisikan struktur Point yang menyimpan koordinat titik, yaitu x dan y. Struktur ini digunakan baik sebagai pusat lingkaran maupun titik yang diuji. File `point.cpp` hanya berisi include karena tidak ada fungsi yang perlu diimplementasikan.

```
MODUL 1 Struct dan Pointer > h point.h > ...
#ifndef POINT_H
#define POINT_H

struct Point {
    double x;
    double y;
};


#endif
```

Gambar 6. point.h

File `circle.h` mendefinisikan struktur `Circle` yang terdiri dari pusat dan jari-jari. Selain itu, dideklarasikan fungsi `distance` untuk menghitung jarak, serta `CheckPointInCircle` untuk menentukan posisi titik.

```
MODUL 1 Struct dan Pointer > h circle.h > ...
#ifndef CIRCLE_H
#define CIRCLE_H

#include "point.h"
#include <string>

struct Circle {
    Point centre;
    double radius;
};

inline constexpr double EPSILON = 1e-9;

double distance(const Circle * c,const Point * p);


std::string CheckPointInCircle(double distance);

#endif
```

Gambar 7. circle.h

Fungsi `distance` diimplementasikan dengan menggunakan rumus jarak Euclidean. Nilai jari-jari disimpan dalam variabel statis agar dapat digunakan oleh fungsi `CheckPointInCircle`, karena fungsi tersebut hanya menerima parameter berupa jarak.

Fungsi `CheckPointInCircle` membandingkan nilai jarak dengan jari-jari. Jika selisihnya mendekati nol, maka titik berada pada lingkaran. Jika jarak lebih kecil, titik berada di dalam, dan jika lebih besar, titik berada di luar.

File `circle.cpp` berisi implementasi dari fungsi-fungsi yang telah dideklarasikan pada `circle.h`, yaitu fungsi `distance` dan `CheckPointInCircle`.

```
MODUL 1 Struct dan Pointer > circle.cpp > CheckPointInCircle(double)
#include "circle.h"
#include <cmath>

static double savedRadius;

double distance(const Circle * c,const Point * p) {
    savedRadius = c->radius;

    return std::sqrt(
        (p->x - c->centre.x) * (p->x - c->centre.x) +
        (p->y - c->centre.y) * (p->y - c->centre.y)
    );
}

std::string CheckPointInCircle(double d) {
    if (std::abs(d - savedRadius) < EPSILON)
        return "On Circle";
    else if (d < savedRadius)
        return "Inside";
    else
        return "Outside";
}
```

Gambar 8. circle.cpp

Fungsi `distance` digunakan untuk menghitung jarak antara titik yang diuji dengan pusat lingkaran. Perhitungan dilakukan menggunakan rumus jarak Euclidean, yaitu:

$$d = \sqrt{(x_p - x_c)^2 + (y_p - y_c)^2}$$

Hasil dari fungsi ini adalah jarak antara titik dan pusat lingkaran. Selain itu, di dalam fungsi ini juga disimpan nilai jari-jari lingkaran ke dalam variabel sementara (static). Ini dilakukan supaya fungsi lain tetap bisa menggunakan nilai radius tanpa perlu mengubah parameter fungsi.

Fungsi kedua, yaitu `CheckPointInCircle`, digunakan untuk menentukan posisi titik berdasarkan jarak yang sudah dihitung tadi. Fungsi ini akan membandingkan jarak tersebut dengan jari-jari lingkaran.

Kalau jaraknya hampir sama dengan jari-jari (menggunakan toleransi kecil atau EPSILON), maka titik berada tepat di lingkaran (**On Circle**). Kalau jaraknya lebih kecil, berarti titik ada di dalam lingkaran (**Inside**). Kalau lebih besar, berarti titik ada di luar (**Outside**).

Penggunaan variabel static di sini sebenarnya hanya untuk menyimpan sementara nilai radius agar bisa dipakai oleh fungsi lain. Cara ini dipakai karena fungsi `CheckPointInCircle` hanya menerima satu parameter, jadi tidak bisa langsung menerima nilai radius.

Secara keseluruhan, file `circle.cpp` ini bertugas melakukan perhitungan utama dan menentukan hasil akhir posisi titik terhadap lingkaran.

Terakhir ada file prob2.cpp, bisa dilihat dari source code tadi.

File `prob2.cpp` merupakan program utama yang menjalankan seluruh proses dalam program. Di dalam file ini terdapat fungsi `main()` yang menjadi titik awal eksekusi program.

Pada bagian awal, program mendeklarasikan dua variabel, yaitu `Circle c` untuk menyimpan data lingkaran (pusat dan jari-jari), serta `Point p` untuk menyimpan koordinat titik yang akan diuji.

Program kemudian menggunakan perulangan `while` untuk membaca input dari pengguna. Input yang dibaca terdiri dari koordinat pusat lingkaran, nilai jari-jari, serta koordinat titik. Dengan menggunakan perulangan ini, program dapat memproses beberapa data sekaligus selama masih ada input yang diberikan.

Setelah data dibaca, program memanggil fungsi `distance` dengan mengirimkan alamat variabel lingkaran dan titik. Fungsi ini akan menghitung jarak antara titik dan pusat lingkaran, lalu mengembalikan hasilnya dalam bentuk nilai numerik.

Nilai jarak tersebut kemudian dikirim ke fungsi `CheckPointInCircle` untuk menentukan posisi titik terhadap lingkaran. Fungsi ini akan mengembalikan hasil berupa teks, yaitu “Inside”, “On Circle”, atau “Outside”.

Hasil yang diperoleh kemudian ditampilkan ke layar menggunakan `std::cout`. Setiap hasil ditampilkan pada baris baru agar mudah dibaca.

Program akan terus berjalan selama masih ada input yang diberikan. Jika input berhenti, maka perulangan akan selesai dan program akan berakhir dengan mengembalikan nilai 0 sebagai tanda bahwa program berjalan dengan sukses.

Lalu selanjutnya, semua kode tadi dijalankan perintah:

```
g++ -std=c++23 -Wall -Wextra -o prob1.exe prob2.cpp circle.cpp point.cpp
```

g++ : Ini adalah compiler C++ buat ngubah kode .cpp jadi program yang bisa dijalankan.

-std=c++23 : pakai standar C++ versi terbaru (C++23)

-Wall : Aktifin semua warning dasar

Contoh: variabel gak kepakai, typo kecil, potensi bug

-Wextra : Tambahan warning lebih detail

-o prob1.exe : hasil compile bakal jadi file bernama prob1.exe dan di sini lah tadi programnya dijalankan

prob1.cpp circle.cpp point.cpp : Ini semua file tadi yang mau dicompile

TAUTAN GIT

https://github.com/RyuuZev/PRAKTIKUM-AlgoritmaStrukturData-MuhammadKurniawanPasya